# HOTEL MANAGEMENT SYSTEM

Project Report for BACSE202 Lab Component Project Assessment 8

Submitted by
Shraddha Sidhan
Registration Number: 25BDS0110
B.Tech CSE - Data Science
Academic Year: 2026-27

School of Computer Science and Engineering SCOPE
Vellore Institute of Technology Vellore

# Table of Contents

1 Case Study 1 ..... 3
2 Abstract ..... 3
3 System Design ..... 3
4 E R Diagram ..... 3
5 Workflow ..... 4
6 Introduction ..... 5
7 Background of the Project ..... 5
8 Hardware and Software Requirements ..... 6
9 Description of Database Design ..... 6
9.1 Entity Summary ..... 6
9.2 Attributes Constraints and Relationships ..... 6
9.3 Normalization to Third Normal Form ..... 6
10 SQL Implementation ..... 7
10.1 CREATE TABLE with Constraints ..... 8
10.2 INSERT Statements Up To Five Sample Records Per Table ..... 9
10.3 DELETE and UPDATE Operations ..... 10
10.4 SELECT Functions Aggregate GROUP BY and HAVING ..... 10
10.5 Subqueries JOIN Queries and Management Information ..... 10
11 Conclusion ..... 10
12 References ..... 10

# 1 Case Study 1

Hotels coordinate guests, rooms, employees, payments and chargeable services daily. Manual registers and disconnected spreadsheets make availability difficult to verify, encourage repeated entry and slow reporting. The Hotel Management System is a relational database solution for connecting these operations.

# 2 Abstract

This PostgreSQL 17 project models nine entities: GUEST, ROOM_TYPE, ROOM, BOOKING, PAYMENT, DEPARTMENT, EMPLOYEE, SERVICE and SERVICE_REQUEST. Primary and foreign keys preserve relationships; CHECK and UNIQUE constraints protect integrity. The design is normalized to Third Normal Form.

# 3 System Design

The system uses a user interface for transactions, an application layer for validation, and PostgreSQL storage. Modules cover guest and room master data, booking, payment, employee, departments, services and management reports. Relationships are represented with foreign keys and the booking-service association is recorded by SERVICE_REQUEST.

# 4 E R Diagram

Figure 1 includes all nine entities, primary keys, foreign keys, relationship names and cardinalities. EMPLOYEE has the recursive manager relationship; SERVICE_REQUEST is the associative entity.

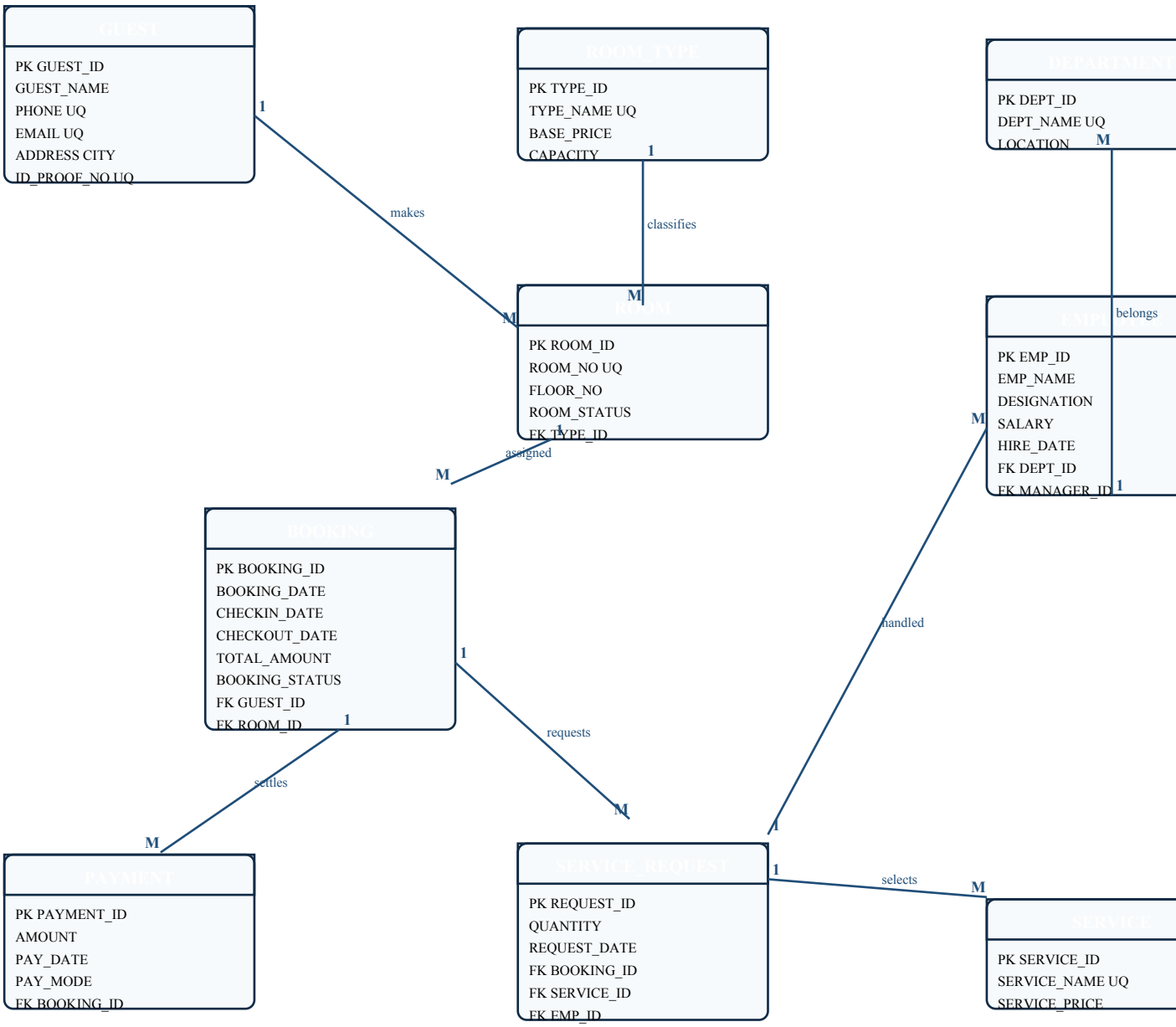



Figure 1 Entity relationship diagram for the Hotel Management System

# 5 Workflow

Booking proceeds only when a suitable room is available. Service requests are recorded during the stay; billing, payment and checkout complete the process.

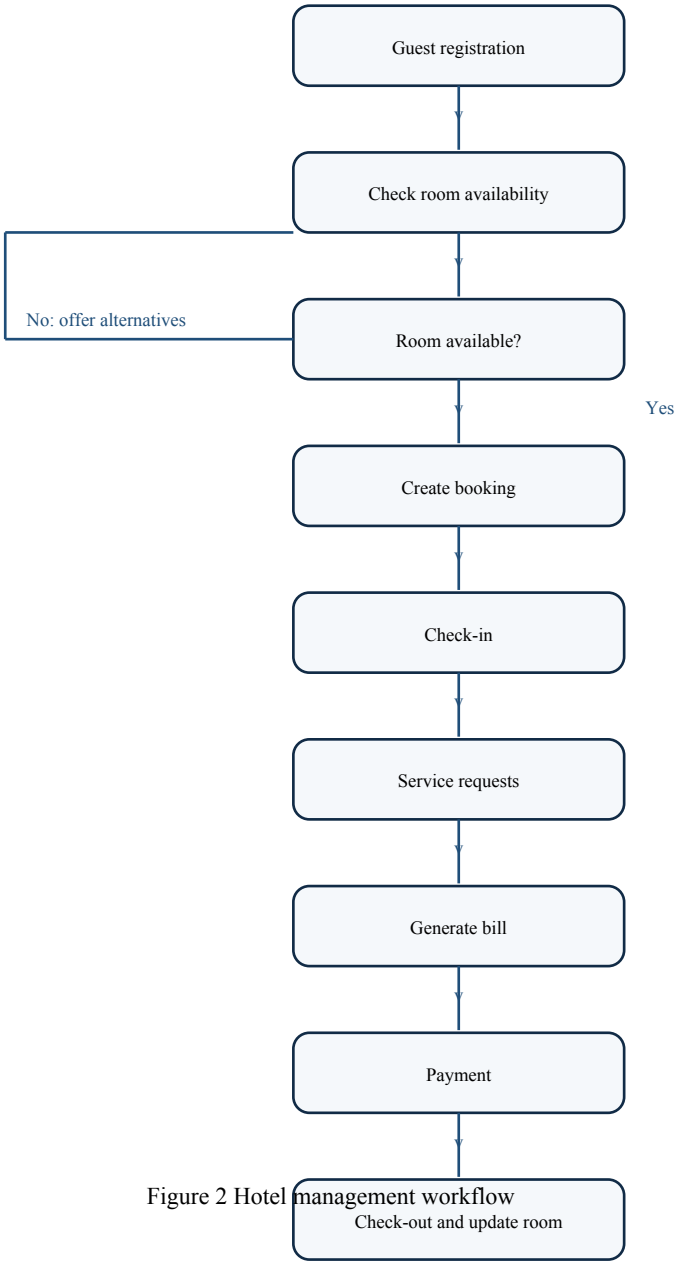



Figure 2 Hotel management workflow

# 6 Introduction

Database management is central to hotel operations because each guest interaction is linked to room inventory, time, money and staff responsibility. A relational database lets authorised users locate a booking, assigned room, service requests and payments from the same controlled data set.

# 7 Background of the Project

Manual registers and spreadsheets can create duplicated entry, weak referential control and difficult audit trails. The proposed PostgreSQL design stores master facts once and uses foreign keys and domain constraints for transactions.

# 8 Hardware and Software Requirements

| Component | Specification / Choice |
|---|---|
| Processor | Intel Core i3 or equivalent |
| Memory | 4 GB RAM or higher |
| Storage | 1 GB free disk space or higher |
| Operating system | Windows Linux or macOS |
| Database server | PostgreSQL 17 |
| Language | SQL |
| Development tools | PostgreSQL client tools and text editor or IDE |

The project schema uses PostgreSQL types and constraints including NUMERIC, DATE, VARCHAR, keys and CHECK clauses.

# 9 Description of Database Design

## 9.1 Entity Summary

| Table | Purpose | Primary Key | References |
|---|---|---|---|
| GUEST | Guest contact identity | GUEST_ID | - |
| ROOM_TYPE | Room category price capacity | TYPE_ID | - |
| ROOM | Individual room status | ROOM_ID | TYPE_ID |
| BOOKING | Reservation and stay | BOOKING_ID | GUEST_ID ROOM_ID |
| PAYMENT | Payment against booking | PAYMENT_ID | BOOKING_ID |
| DEPARTMENT | Department data | DEPT_ID | - |
| EMPLOYEE | Employee hierarchy | EMP_ID | DEPT_ID MANAGER_ID |
| SERVICE | Service catalogue | SERVICE_ID | - |
| SERVICE_REQUEST | Services supplied | REQUEST_ID | BOOKING_ID SERVICE_ID EMP_ID |

## 9.2 Attributes Constraints and Relationships

GUEST has unique phone, email and identity-proof values. ROOM_TYPE uses positive price and capacity; ROOM accepts Available, Occupied or Maintenance. BOOKING connects guest and room, requires checkout after check-in, and restricts statuses. PAYMENT requires a positive amount. EMPLOYEE belongs to a department and may refer to another employee as manager. SERVICE_REQUEST connects booking, service and employee with a positive quantity.

## 9.3 Normalization to Third Normal Form

1NF: attributes are atomic and every table has a primary key. 2NF: non-key fields depend on the whole key. 3NF: room type, department and service price facts reside in separate tables, preventing transitive dependencies and update anomalies.

# 10 SQL Implementation

The following listings preserve available project schema and sample data. No output screenshots are included because none were supplied, and no unexecuted query results are claimed.

## 10.1 CREATE TABLE with Constraints

```
-- Hotel Management System — schema
-- Table definitions are taken unchanged from the project report (section 7.1).
-- IF NOT EXISTS only makes this script safe to re-run; it never alters or drops anything.

CREATE TABLE IF NOT EXISTS GUEST (
    GUEST_ID INT PRIMARY KEY,
    GUEST_NAME VARCHAR(50) NOT NULL,
    PHONE VARCHAR(15) UNIQUE NOT NULL,
    EMAIL VARCHAR(100) UNIQUE,
    ADDRESS VARCHAR(100),
    CITY VARCHAR(50),
    ID_PROOF_NO VARCHAR(30) UNIQUE NOT NULL
);

CREATE TABLE IF NOT EXISTS ROOM_TYPE (
    TYPE_ID INT PRIMARY KEY,
    TYPE_NAME VARCHAR(30) UNIQUE NOT NULL,
    BASE_PRICE NUMERIC(10,2) NOT NULL CHECK (BASE_PRICE > 0),
    CAPACITY INT NOT NULL CHECK (CAPACITY > 0)
);

CREATE TABLE IF NOT EXISTS ROOM (
    ROOM_ID INT PRIMARY KEY,
    ROOM_NO INT UNIQUE NOT NULL,
    FLOOR_NO INT NOT NULL CHECK (FLOOR_NO > 0),
    ROOM_STATUS VARCHAR(20) DEFAULT 'Available'
        CHECK (ROOM_STATUS IN ('Available', 'Occupied', 'Maintenance')),
    TYPE_ID INT NOT NULL,
    FOREIGN KEY (TYPE_ID) REFERENCES ROOM_TYPE(TYPE_ID)
);

CREATE TABLE IF NOT EXISTS DEPARTMENT (
    DEPT_ID INT PRIMARY KEY,
    DEPT_NAME VARCHAR(50) UNIQUE NOT NULL,
    LOCATION VARCHAR(50)
);

CREATE TABLE IF NOT EXISTS EMPLOYEE (
    EMP_ID INT PRIMARY KEY,
    EMP_NAME VARCHAR(50) NOT NULL,
    DESIGNATION VARCHAR(50),
    SALARY NUMERIC(10,2) CHECK (SALARY > 0),
    HIRE_DATE DATE NOT NULL,
    DEPT_ID INT NOT NULL,
    MANAGER_ID INT,
    FOREIGN KEY (DEPT_ID) REFERENCES DEPARTMENT(DEPT_ID),
    FOREIGN KEY (MANAGER_ID) REFERENCES EMPLOYEE(EMP_ID)
);

CREATE TABLE IF NOT EXISTS BOOKING (
    BOOKING_ID INT PRIMARY KEY,
    BOOKING_DATE DATE NOT NULL,
    CHECKIN_DATE DATE NOT NULL,
    CHECKOUT_DATE DATE NOT NULL,
    TOTAL_AMOUNT NUMERIC(10,2) CHECK (TOTAL_AMOUNT >= 0),
    BOOKING_STATUS VARCHAR(20) DEFAULT 'Confirmed'
        CHECK (BOOKING_STATUS IN ('Confirmed', 'Checked-In', 'Checked-Out', 'Cancelled')),
    GUEST_ID INT NOT NULL,
    ROOM_ID INT NOT NULL,
    FOREIGN KEY (GUEST_ID) REFERENCES GUEST(GUEST_ID),
    FOREIGN KEY (ROOM_ID) REFERENCES ROOM(ROOM_ID),
    CHECK (CHECKOUT_DATE > CHECKIN_DATE)
);

CREATE TABLE IF NOT EXISTS PAYMENT (
    PAYMENT_ID INT PRIMARY KEY,
    AMOUNT NUMERIC(10,2) NOT NULL CHECK (AMOUNT > 0),
    PAY_DATE DATE NOT NULL,
    PAY_MODE VARCHAR(20) CHECK (PAY_MODE IN ('Cash', 'Card', 'UPI', 'Net Banking')),
    BOOKING_ID INT NOT NULL,
    FOREIGN KEY (BOOKING_ID) REFERENCES BOOKING(BOOKING_ID)
);

CREATE TABLE IF NOT EXISTS SERVICE (
    SERVICE_ID INT PRIMARY KEY,
    SERVICE_NAME VARCHAR(50) UNIQUE NOT NULL,
    SERVICE_PRICE NUMERIC(10,2) NOT NULL CHECK (SERVICE_PRICE > 0)
);

CREATE TABLE IF NOT EXISTS SERVICE_REQUEST (
    REQUEST_ID INT PRIMARY KEY,
```

```
    QUANTITY INT NOT NULL CHECK (QUANTITY > 0),
    REQUEST_DATE DATE NOT NULL,
    BOOKING_ID INT NOT NULL,
    SERVICE_ID INT NOT NULL,
    EMP_ID INT NOT NULL,
    FOREIGN KEY (BOOKING_ID) REFERENCES BOOKING(BOOKING_ID),
    FOREIGN KEY (SERVICE_ID) REFERENCES SERVICE(SERVICE_ID),
    FOREIGN KEY (EMP_ID) REFERENCES EMPLOYEE(EMP_ID)
);
```

## 10.2 INSERT Statements Up To Five Sample Records Per Table

```
-- Hotel Management System — starting records
-- These are the five rows per table inserted in the project report (section 7.2).
-- setup.sh loads this file only when the database is completely empty.

INSERT INTO GUEST VALUES
(1, 'Rahul Sharma', '9876543210', 'rahul@gmail.com', 'MG Road', 'Bangalore', 'AADHAR001'),
(2, 'Priya Nair', '9876543211', 'priya@gmail.com', 'Anna Nagar', 'Chennai', 'AADHAR002'),
(3, 'Arjun Mehta', '9876543212', 'arjun@gmail.com', 'Kothrud', 'Pune', 'AADHAR003'),
(4, 'Sneha Reddy', '9876543213', 'sneha@gmail.com', 'Banjara Hills', 'Hyderabad', 'AADHAR004'),
(5, 'Karan Singh', '9876543214', 'karan@gmail.com', 'Sector 17', 'Chandigarh', 'AADHAR005');

INSERT INTO ROOM_TYPE VALUES
(1, 'Standard', 2500, 2),
(2, 'Deluxe', 4000, 3),
(3, 'Suite', 7000, 4),
(4, 'Presidential', 12000, 6),
(5, 'Family', 5500, 5);

INSERT INTO ROOM VALUES
(1, 101, 1, 'Available', 1),
(2, 102, 1, 'Occupied', 2),
(3, 201, 2, 'Available', 3),
(4, 301, 3, 'Maintenance', 4),
(5, 302, 3, 'Available', 5);

INSERT INTO DEPARTMENT VALUES
(1, 'Front Office', 'Ground Floor'),
(2, 'Housekeeping', 'First Floor'),
(3, 'Food and Beverage', 'Ground Floor'),
(4, 'Human Resources', 'Second Floor'),
(5, 'Maintenance', 'Basement');

INSERT INTO EMPLOYEE VALUES
(1, 'Raj Kumar', 'General Manager', 90000, '2020-01-10', 1, NULL),
(2, 'Anita Rao', 'Front Office Manager', 60000, '2021-03-15', 1, 1),
(3, 'Vikram Das', 'Housekeeping Manager', 55000, '2021-06-20', 2, 1),
(4, 'Meena Shah', 'HR Manager', 65000, '2022-02-10', 4, 1),
(5, 'Ravi Kumar', 'Service Executive', 35000, '2023-07-05', 3, 3);

INSERT INTO BOOKING VALUES
(1, '2026-08-01', '2026-08-05', '2026-08-07', 8000, 'Checked-Out', 1, 2),
(2, '2026-08-02', '2026-08-08', '2026-08-10', 14000, 'Checked-Out', 2, 3),
(3, '2026-08-05', '2026-08-12', '2026-08-15', 7500, 'Confirmed', 3, 1),
(4, '2026-08-07', '2026-08-18', '2026-08-20', 24000, 'Checked-In', 4, 4),
(5, '2026-08-10', '2026-08-22', '2026-08-25', 16500, 'Confirmed', 5, 5);

INSERT INTO PAYMENT VALUES
(1, 4000, '2026-08-05', 'UPI', 1),
(2, 4000, '2026-08-07', 'Card', 1),
(3, 7000, '2026-08-08', 'Cash', 2),
(4, 7000, '2026-08-10', 'UPI', 2),
(5, 7500, '2026-08-12', 'Card', 3);

INSERT INTO SERVICE VALUES
(1, 'Room Service', 500),
(2, 'Laundry', 300),
(3, 'Spa', 1500),
(4, 'Airport Pickup', 1000),
(5, 'Conference Hall', 5000);

INSERT INTO SERVICE_REQUEST VALUES
(1, 2, '2026-08-06', 1, 1, 5),
(2, 3, '2026-08-06', 1, 2, 5),
(3, 1, '2026-08-09', 2, 3, 5),
(4, 2, '2026-08-13', 3, 4, 5),
(5, 1, '2026-08-19', 4, 5, 5);
```

### 10.3 DELETE and UPDATE Operations

```
DELETE FROM SERVICE_REQUEST WHERE REQUEST_ID=5;
DELETE FROM PAYMENT WHERE PAYMENT_ID=5;
DELETE FROM BOOKING WHERE BOOKING_ID=5;
DELETE FROM ROOM WHERE ROOM_ID=5; DELETE FROM GUEST WHERE GUEST_ID=5;
DELETE FROM SERVICE WHERE SERVICE_ID=5; DELETE FROM EMPLOYEE WHERE EMP_ID=5;
DELETE FROM DEPARTMENT WHERE DEPT_ID=5; DELETE FROM ROOM_TYPE WHERE TYPE_ID=5;
UPDATE GUEST SET CITY='Bengaluru' WHERE GUEST_ID=1;
UPDATE ROOM_TYPE SET BASE_PRICE=2600 WHERE TYPE_ID=1; UPDATE ROOM SET ROOM_STATUS='Occupied' WHERE ROOM_ID=1;
UPDATE BOOKING SET BOOKING_STATUS='Checked-In' WHERE BOOKING_ID=3; UPDATE PAYMENT SET PAY_MODE='Card' WHERE PAYMENT_ID=1;
UPDATE DEPARTMENT SET LOCATION='Ground Floor' WHERE DEPT_ID=1; UPDATE EMPLOYEE SET SALARY=92000 WHERE EMP_ID=1;
UPDATE SERVICE SET SERVICE_PRICE=550 WHERE SERVICE_ID=1; UPDATE SERVICE_REQUEST SET QUANTITY=3 WHERE REQUEST_ID=1;
```

### 10.4 SELECT Functions Aggregate GROUP BY and HAVING

```
SELECT * FROM GUEST;
SELECT GUEST_NAME, UPPER(CITY), LENGTH(GUEST_NAME) FROM GUEST;
SELECT COUNT(*) FROM GUEST; SELECT AVG(SALARY), MAX(SALARY) FROM EMPLOYEE;
SELECT PAY_MODE, COUNT(*), SUM(AMOUNT) FROM PAYMENT GROUP BY PAY_MODE;
SELECT d.DEPT_NAME, AVG(e.SALARY) FROM DEPARTMENT d JOIN EMPLOYEE e ON e.DEPT_ID=d.DEPT_ID GROUP BY d.DEPT_NAME HAVING AVG(e.SA
```

### 10.5 Subqueries JOIN Queries and Management Information

```
SELECT EMP_NAME, SALARY FROM EMPLOYEE WHERE SALARY>(SELECT AVG(SALARY) FROM EMPLOYEE);
SELECT b.BOOKING_ID,g.GUEST_NAME,r.ROOM_NO,rt.TYPE_NAME FROM BOOKING b JOIN GUEST g ON g.GUEST_ID=b.GUEST_ID JOIN ROOM r ON r.R
SELECT rt.TYPE_NAME,COUNT(b.BOOKING_ID),COALESCE(SUM(b.TOTAL_AMOUNT),0) FROM ROOM_TYPE rt LEFT JOIN ROOM r ON r.TYPE_ID=rt.TYPE
```

## 11 Conclusion

The nine-table design separates master information from transactions, enforces relationships and supports management reporting. Future enhancements can add online reservations, role-based access, invoices, audit logs and real-time housekeeping status.

## 12 References


[1] PostgreSQL Global Development Group, PostgreSQL 17 Documentation, https://www.postgresql.org/docs/17/ (accessed 12 October 2026).

[2] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2016.

[3] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed., McGraw Hill, 2019.

[4] PostgreSQL Global Development Group, SQL Language Documentation, https://www.postgresql.org/docs/current/sql.html (accessed 12 October 2026).